---

1. Producer-Consumer Problem

- A classic multithreading problem where producers generate data and place it into a shared buffer, while consumers retrieve and process the data.
- Can be solved using `BlockingQueue`, `wait-notify`, or `Semaphore`.

This solution ensures thread safety and proper synchronization using `BlockingQueue`.

```
import java.util.concurrent.BlockingQueue;
import java.util.concurrent.LinkedBlockingQueue;

class Producer implements Runnable {
    private final BlockingQueue<Integer> queue;
    private static int count = 0;

    public Producer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        try {
            while (true) {
                Thread.sleep(1000); // Simulate production delay
                queue.put(++count);
                System.out.println("Produced: " + count);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```java
class Consumer implements Runnable {
    private final BlockingQueue<Integer> queue;

    public Consumer(BlockingQueue<Integer> queue) {
        this.queue = queue;
    }

    @Override
    public void run() {
        try {
            while (true) {
                Thread.sleep(1500); // Simulate consumption delay
                int item = queue.take();
                System.out.println("Consumed: " + item);
            }
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

```java
public class ProducerConsumerExample {
    public static void main(String[] args) {
        BlockingQueue<Integer> queue = new LinkedBlockingQueue<>(5);
        Thread producer = new Thread(new Producer(queue));
        Thread consumer = new Thread(new Consumer(queue));

        producer.start();
        consumer.start();
    }
}
```

- **Ensures thread safety with `BlockingQueue`**
- **Handles synchronization automatically**

---

2. Deadlock Example

- Occurs when two or more threads hold resources and wait for each other indefinitely.
- Can be demonstrated using `synchronized` blocks on nested resources.
- Solutions: Avoid circular wait, use `tryLock()`, or implement a resource hierarchy.

Demonstrates a scenario where two threads wait on each other indefinitely.

```
class Resource {
    public void methodA(Resource other) {
        synchronized (this) {
            System.out.println(Thread.currentThread().getName() + " locked " + this);
            try { Thread.sleep(100); } catch (InterruptedException ignored) {}
            synchronized (other) {
                System.out.println(Thread.currentThread().getName() + " locked " + otl
            }
        }
    }
}

public class DeadlockExample {
    public static void main(String[] args) {
        Resource r1 = new Resource();
        Resource r2 = new Resource();

        Thread t1 = new Thread(() -> r1.methodA(r2), "Thread-1");
        Thread t2 = new Thread(() -> r2.methodA(r1), "Thread-2");

        t1.start();
        t2.start();
    }
}
```

🔹 **Fix:** Use `tryLock()` from `ReentrantLock` or a resource hierarchy to prevent circular waits.

---

3. FIFO Cache (Using LinkedHashMap)

- A simple caching mechanism where the oldest element is removed when the cache is full.
- Implemented using `LinkedHashMap` or `Queue`.

Implements a FIFO cache by removing the oldest entry when capacity is exceeded.

```java
import java.util.*;

class FIFOCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public FIFOCache(int capacity) {
        super(capacity, 1.0f, false);
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

public class FIFOCacheExample {
    public static void main(String[] args) {
        FIFOCache<Integer, String> cache = new FIFOCache<>(3);
        cache.put(1, "A");
        cache.put(2, "B");
        cache.put(3, "C");
        System.out.println(cache); // {1=A, 2=B, 3=C}

        cache.put(4, "D"); // Removes 1
        System.out.println(cache); // {2=B, 3=C, 4=D}
    }
}
```

🔹 **Uses `LinkedHashMap` with access order set to `false`**

---

4. LRU Cache (Using LinkedHashMap)

- Removes the least recently accessed element when the cache reaches its capacity.
- Can be efficiently implemented using `LinkedHashMap` with access order enabled, or a combination of `HashMap` and `DoublyLinkedList`.

Removes the **Least Recently Used** element when full.

```java
import java.util.*;

class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LRUCache(int capacity) {
        super(capacity, 1.0f, true); // Access-order enabled
        this.capacity = capacity;
    }

    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }
}

public class LRUCacheExample {
    public static void main(String[] args) {
        LRUCache<Integer, String> cache = new LRUCache<>(3);
        cache.put(1, "A");
        cache.put(2, "B");
        cache.put(3, "C");
        System.out.println(cache); // {1=A, 2=B, 3=C}

        cache.get(1); // Access 1
        cache.put(4, "D"); // Removes 2 (Least Recently Used)
        System.out.println(cache); // {3=C, 1=A, 4=D}
    }
}
```

- Uses `LinkedHashMap` with access order enabled (`true`)
- Efficiently updates access order on `get()`

Summary

Problem	Solution Used	Key Features
Producer-Consumer	`BlockingQueue`	Automatic synchronization
Deadlock	`synchronized`	Demonstrates circular wait
FIFO Cache	`LinkedHashMap`	Removes oldest entry
LRU Cache	`LinkedHashMap` (access order)	Removes least recently used
